

7 ПРАКТИЧЕСКАЯ РАБОТА (НАПОЛНЕНИЕ БАЗ ДАННЫХ И РАЗРАБОТКА SQL-ЗАПРОСОВ)

7.1 Цель работы

Целью практической работы является формирование навыков подготовки репрезентативного набора тестовых данных и разработки содержательных SQL-запросов, обеспечивающих извлечение, объединение, фильтрацию, группировку, агрегирование и анализ информации, хранящейся в разработанной реляционной базе данных.

7.2 Задачи работы

В процессе выполнения практической работы необходимо:

- проанализировать физическую структуру базы данных, разработанную в ПР № 6;
- определить состав тестовых данных, необходимых для проверки основных сценариев системы;
- сформировать взаимосвязанный и непротиворечивый набор тестовых записей;
- разработать воспроизводимый SQL-скрипт наполнения базы данных;
- проверить соблюдение реализованных ограничений целостности;
- определить информационные потребности пользователей системы;

- сформулировать прикладные задачи, решаемые посредством SQL-запросов;
- разработать запросы к одной и нескольким таблицам;
- использовать фильтрацию, сортировку, соединение и агрегирование данных;
- реализовать групповые запросы;
- реализовать запросы с условиями над агрегированными результатами;
- использовать вложенные запросы и/или общие табличные выражения;
- проверить корректность результатов выполнения запросов;
- оформить SQL-запросы в виде отдельного воспроизводимого скрипта;
- подготовить базу данных к последующей реализации функций, процедур и триггеров.

7.3 Теоретические сведения

После разработки физической структуры базы данных необходимо обеспечить её содержательное наполнение.

Пустая база данных позволяет проверить корректность структуры, однако не позволяет в достаточной степени оценить:

- правильность связей;
- корректность SQL-запросов;
- поведение ограничений;

- обработку различных состояний объектов;
- работу агрегирующих операций;
- обработку граничных случаев;
- корректность последующей серверной логики;
- работу пользовательского интерфейса.

Поэтому тестовые данные являются самостоятельным элементом проектирования и тестирования системы.

Набор тестовых данных должен быть сформирован таким образом, чтобы представлять не один типовой сценарий, а различные допустимые состояния предметной области.

Например, если система содержит заявки, тестовый набор может включать:

- новые заявки;
- заявки в работе;
- завершённые заявки;
- просроченные заявки;
- заявки разных категорий;
- заявки разных пользователей и исполнителей.

Это позволяет формировать содержательные запросы и проверять поведение системы в различных условиях.

7.3.1 Требования к тестовым данным

Тестовые данные должны:

- соответствовать структуре базы данных;
- соблюдать ограничения целостности;

- содержать взаимосвязанные записи;
- включать различные значения категориальных признаков;
- включать различные состояния объектов;
- обеспечивать получение неодинаковых результатов при фильтрации;
- обеспечивать возможность группировки и агрегирования;
- содержать данные для проверки связей между таблицами;
- обеспечивать выполнение основных пользовательских сценариев.

Недостаточным является набор данных, в котором все записи практически идентичны.

Например, если все заказы имеют:

- одного клиента;
- одну дату;
- один статус;
- один товар;

подобный набор не позволяет полноценно проверить большинство запросов.

Тестовые данные могут быть условными и не обязаны соответствовать реально существующим лицам, организациям или объектам.

Однако они должны быть логически согласованными.

Например, нежелательны ситуации:

- дата завершения операции предшествует дате её создания;
- оплаченный заказ не содержит ни одной позиции;
- сотрудник назначен в подразделение, которого не существует;
- ссылка ведёт на отсутствующий объект;
- статус объекта противоречит заполненным данным;
- числовые характеристики выходят за допустимые диапазоны.

Смысловая непротиворечивость данных необходима не только для корректного выполнения настоящей работы, но и для последующих этапов разработки и тестирования.

Все тестовые данные должны быть представлены в воспроизводимом SQL-скрипте.

Основной файл:

```
02_insert_data.sql
```

должен содержать инструкции, позволяющие заполнить ранее созданную структуру базы данных.

Наполнение базы вручную исключительно посредством графического интерфейса СУБД не считается достаточным.

После выполнения:

```
01_create_tables.sql
```

и затем:

```
02_insert_data.sql
```

должна быть получена база данных, содержащая структуру и полный тестовый набор, необходимый для дальнейшей работы.

При заполнении взаимосвязанных таблиц необходимо учитывать ограничения внешних ключей.

Родительская запись должна существовать до создания зависимой записи, если связь не допускает отсутствия соответствующего объекта.

Например, до создания заказа должны существовать:

- клиент;
- необходимые справочные значения;
- при необходимости — сотрудник или иные связанные объекты.

После этого могут создаваться:

- заказ;
- позиции заказа;
- платежи;
- иные зависимые записи.

Следовательно, порядок команд INSERT должен соответствовать структуре зависимостей базы данных.

7.3.2 Назначение SQL-запросов

SQL-запрос в рамках настоящей практической работы должен решать конкретную информационную задачу.

Недостаточно продемонстрировать знание синтаксической конструкции.

Например:

```
SELECT *  
FROM users;
```

является корректным SQL-запросом, однако сам по себе не демонстрирует решение содержательной задачи и не может рассматриваться как сложный прикладной запрос.

Предпочтительная постановка задачи:

Получить список активных пользователей определённой роли, зарегистрированных после указанной даты, отсортированный по дате регистрации.

SQL-запрос должен являться реализацией предварительно сформулированной прикладной задачи.

7.3.3 Простая выборка и фильтрация

Оператор `SELECT` используется для извлечения данных из одной или нескольких таблиц.

При формировании выборки рекомендуется указывать только необходимые столбцы.

Использование:

```
SELECT *
```

допускается для диагностических операций и предварительного просмотра данных, однако не должно составлять основу набора прикладных запросов.

Фильтрация данных выполняется посредством условий, например:

```
WHERE
```

Условия могут использовать:

- сравнение;
- логические операции;
- проверку диапазона;
- проверку принадлежности множеству;
- поиск по шаблону;
- проверку наличия или отсутствия значения.

Фильтрация должна соответствовать конкретной задаче предметной области.

7.3.4 Сортировка

Оператор `ORDER BY` используется для упорядочивания результата по одному или нескольким признакам.

Сортировка может применяться:

- по возрастанию;
- по убыванию;
- по нескольким столбцам.

Например, для информационной системы может потребоваться получить:

- заявки по сроку выполнения;
- товары по стоимости;
- пользователей по дате регистрации;
- операции по времени выполнения.

Сортировка должна использоваться там, где порядок результата имеет прикладное значение.

7.3.5 Соединение таблиц

Данные реальной реляционной системы обычно распределены между несколькими связанными таблицами.

Для получения комплексной информации используются операции соединения.

Наиболее часто применяется INNER JOIN, позволяющий объединить строки при выполнении заданного условия.

Кроме того, в зависимости от задачи могут использоваться внешние соединения.

Например, запрос может объединять:

- клиента и его заказы;
- заявку и исполнителя;
- заказ и позиции заказа;

- документ и автора;
- сотрудника и подразделение.

Соединение должно выполняться по логически обоснованным условиям, соответствующим отношениям физической модели.

Не допускается использование соединения таблиц без содержательной связи исключительно для усложнения запроса.

7.3.6 Агрегатные функции

Агрегатные функции позволяют вычислять характеристики множества строк.

К типовым операциям относятся:

- количество;
- сумма;
- среднее значение;
- минимальное значение;
- максимальное значение.

Агрегирование может применяться, например, для определения:

- количества заказов клиента;
- общей суммы продаж;
- среднего времени выполнения заявок;
- количества объектов определённого состояния;
- максимального значения показателя.

Результат агрегирования должен иметь прикладной смысл.

7.3.7 Группировка

Оператор GROUP BY позволяет объединить строки в группы для последующего вычисления агрегированных показателей.

Например:

- количество заказов по клиентам;
- сумма платежей по месяцам;
- количество заявок по категориям;
- количество сотрудников по подразделениям.

Группировка должна формироваться по признаку, имеющему смысл для анализа рассматриваемых данных.

7.3.8 HAVING

Оператор HAVING используется для ограничения результатов после группировки.

В отличие от WHERE, который ограничивает исходные строки, HAVING применяется к сформированным группам.

Например:

вывести только клиентов, оформивших более пяти заказов.

Или:

вывести категории, среднее значение показателя в которых превышает установленный уровень.

В рамках настоящей практической работы необходимо продемонстрировать понимание различия между WHERE и HAVING.

7.3.9 Вложенные запросы

Вложенный запрос представляет собой запрос, используемый внутри другой SQL-конструкции.

Вложенные запросы могут применяться для:

- определения промежуточного множества;
- сравнения со статистическим показателем;
- проверки существования связанных записей;
- формирования динамического условия.

Например:

получить товары, стоимость которых выше средней стоимости товаров.

Использование вложенного запроса должно быть содержательно оправдано.

7.3.10 Общие табличные выражения

Общее табличное выражение — CTE — позволяет сформировать именованный промежуточный набор данных, который используется основным запросом.

Типовая конструкция:

<pre>WITH ... SELECT ...</pre>

CTE может применяться для:

- повышения читаемости сложного запроса;
- последовательного построения промежуточных результатов;
- рекурсивной обработки иерархий;

- повторного обращения к логически выделенному набору данных.

В рамках настоящей работы допускается использование СТЕ вместо одного из вложенных запросов, если его применение соответствует решаемой задаче.

7.3.11 Параметризация запросов

Прикладные запросы в реальной системе редко работают только с заранее жёстко заданными значениями.

Например, пользователь может выбирать:

- дату;
- клиента;
- категорию;
- статус;
- диапазон стоимости;
- подразделение.

На этапе настоящей практической работы допускается выполнение запросов с конкретными тестовыми значениями.

Однако при их проектировании следует учитывать возможность последующей передачи соответствующих значений из пользовательского интерфейса или серверной функции.

7.3.12 Требования к содержательности SQL-запросов

Запрос учитывается в минимальном количестве только в том случае, если он:

- решает самостоятельную прикладную задачу;
- возвращает содержательный результат;
- отличается от других запросов не только значением одного литерала;
- демонстрирует одну или несколько требуемых конструкций SQL;
- соответствует структуре и требованиям проектируемой системы.

Например, следующие запросы:

- найти пользователя с `id = 1`;
- найти пользователя с `id = 2`;
- найти пользователя с `id = 3`;

рассматриваются как одна логическая задача, а не как три содержательных запроса.

Аналогично простое изменение сортировки одного и того же набора не образует новый самостоятельный прикладной запрос.

7.4 Задание

На основании базы данных, созданной в ПР № 6, необходимо:

- сформировать репрезентативный набор тестовых данных;
- разработать SQL-скрипт наполнения;
- сформулировать не менее десяти самостоятельных информационных задач;
- реализовать для них SQL-запросы;
- проверить корректность получаемых результатов;
- оформить запросы в отдельный SQL-файл.

7.4.1 Наполнение базы данных

Тестовый набор должен обеспечивать выполнение всех разработанных запросов.

Минимальные требования к объёму данных:

- каждая основная таблица должна содержать тестовые записи;
- справочные таблицы должны содержать достаточное количество различных значений для проверки фильтрации;
- основная операционная или транзакционная сущность проекта должна содержать не менее 10 записей;
- база данных в совокупности должна содержать не менее 50 содержательных записей, распределённых между связанными таблицами;
- должны присутствовать различные значения состояний, категорий или иных признаков, если они предусмотрены моделью;
- должны присутствовать данные, позволяющие получать непустые результаты агрегирования и группировки.

Для предметных областей, в которых указанное распределение не соответствует характеру данных, допускается иной объём по согласованию с преподавателем.

Минимум в 50 записей не означает необходимость равномерно распределять их между всеми таблицами.

7.4.2 Обязательные категории SQL-запросов

В составе не менее 10 содержательных запросов должны присутствовать запросы следующих категорий.

Не менее одного запроса с содержательным условием WHERE.

Не менее одного запроса, для которого ORDER BY имеет прикладной смысл.

Не менее одного запроса с JOIN.

Не менее одного запроса, объединяющего данные не менее чем из трёх связанных таблиц.

Не менее одного запроса с применением агрегатной функции.

Не менее одного запроса с GROUP BY.

Не менее одного запроса с HAVING.

Не менее одного запроса с содержательно обоснованным вложенным запросом либо CTE.

Не менее одного запроса, одновременно использующего несколько средств SQL, например:

- несколько JOIN;
- группировку;
- агрегирование;
- фильтрацию;
- сортировку.

Не менее одного запроса, отражающего специфическую информационную потребность выбранной предметной области.

Один сложный запрос может соответствовать нескольким указанным категориям, однако общее количество самостоятельных прикладных задач должно составлять не менее десяти.

7.4.3 Необязательные, но рекомендуемые конструкции

В зависимости от возможностей выбранной СУБД и предметной области рекомендуется дополнительно рассмотреть:

- DISTINCT;
- CASE;
- EXISTS;
- NOT EXISTS;
- операции над множествами;
- оконные функции;
- функции работы с датами;
- строковые функции;
- рекурсивные CTE.

Использование дополнительных конструкций не должно осуществляться исключительно ради формального усложнения SQL-кода.

7.4.4 Оформление файла запросов

Все разработанные запросы необходимо сохранить в файле:

`03_queries.sql`

Каждый запрос должен сопровождаться комментарием, содержащим постановку решаемой задачи.

Рекомендуемый формат:

```
-- Запрос 1.  
-- Получить список ...  
SELECT ...
```

SQL-файл должен быть структурирован таким образом, чтобы преподаватель мог определить:

- назначение запроса;
- используемую SQL-конструкцию;
- ожидаемый результат.

7.4.5 Проверка результатов

Каждый разработанный запрос должен быть выполнен на подготовленном тестовом наборе данных.

Необходимо проверить:

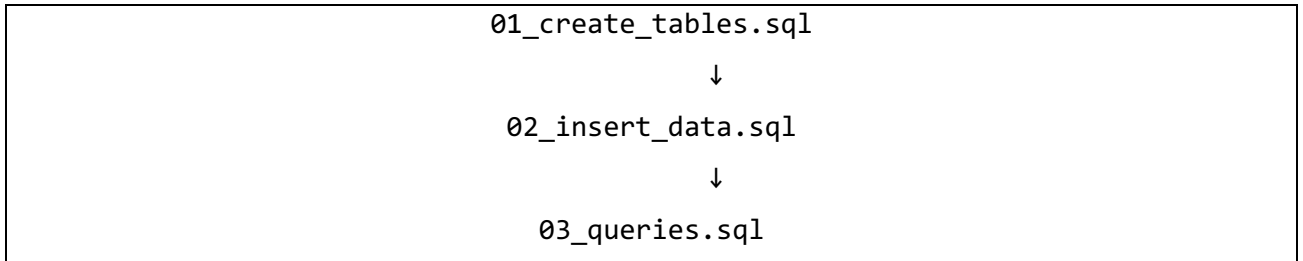
- отсутствие ошибок выполнения;
- соответствие результата сформулированной задаче;
- отсутствие неожиданных дубликатов;
- корректность соединений;
- корректность агрегирования;
- правильность условий фильтрации.

Если запрос возвращает пустой набор исключительно потому, что подходящие тестовые данные отсутствуют, необходимо дополнить тестовый набор.

Исключением являются запросы, специально предназначенные для проверки отсутствия определённых данных.

7.4.6 Проверка воспроизводимости

Полное развёртывание текущей версии базы должно выполняться в следующей последовательности:



После выполнения первых двух файлов должна быть получена база данных, пригодная для выполнения всех запросов третьего файла.

Не допускается зависимость итогового результата от данных, введённых вручную вне SQL-скриптов.

7.5 Порядок выполнения работы

- Проанализировать замечания по ПР № 6.
- Проверить актуальность 01_create_tables.sql.
- Определить необходимые категории тестовых данных.
- Подготовить взаимосвязанный набор записей.
- Сформировать 02_insert_data.sql.
- Развернуть чистую базу данных.
- Выполнить скрипт создания структуры.
- Выполнить скрипт наполнения.
- Проверить ограничения и связи.
- Сформулировать не менее десяти прикладных информационных задач.

- Реализовать простые запросы.
- Реализовать запросы с соединениями.
- Реализовать агрегирующие запросы.
- Реализовать групповые запросы.
- Реализовать запрос с HAVING.
- Реализовать вложенный запрос или CTE.
- Реализовать комплексный запрос.
- Проверить результаты.
- Сформировать 03_queries.sql.
- Проверить полную воспроизводимость проекта.
- Подготовить отчёт.
- Подготовить результаты к защите.

7.6 Содержание отчёта

Отчёт по практической работе должен содержать:

- титульный лист;
- номер и название практической работы;
- цель работы;
- краткое описание принципов формирования тестового набора;
- сведения об объёме данных;
- примеры тестовых данных;
- перечень сформулированных информационных задач;
- SQL-код основных запросов;

- результат выполнения каждого запроса;
- пояснение используемых SQL-конструкций;
- описание наиболее сложного запроса;
- результаты проверки воспроизводимости;
- сведения об изменениях физической модели, если они выполнялись;
- вывод по результатам работы.

В основной текст отчёта рекомендуется включать наиболее значимые запросы.

Полные файлы 02_insert_data.sql и 03_queries.sql предоставляются отдельно.

7.7 Результаты, предоставляемые преподавателю

Для защиты практической работы обучающийся предоставляет:

- отчёт;
- 01_create_tables.sql;
- 02_insert_data.sql;
- 03_queries.sql;
- развёрнутую тестовую базу либо возможность её воспроизвести.

Рекомендуемая структура:

/project
/sql
01_create_tables.sql
02_insert_data.sql
03_queries.sql

Обучающийся должен быть готов:

- развернуть базу на основании SQL-скриптов;
- объяснить логику формирования тестового набора;
- сформулировать назначение любого запроса;
- объяснить используемые соединения;
- объяснить разницу между WHERE и HAVING;
- объяснить использование агрегатных функций;
- объяснить вложенный запрос или CTE;
- изменить условие запроса по требованию преподавателя;
- объяснить результат выполнения запроса.

7.8 Использование результатов в курсовой работе

Тестовый набор данных и SQL-запросы являются частью технической реализации курсовой работы.

Минимальные требования ПР № 7 определяют только объём, достаточный для зачёта практической работы.

В курсовой работе тестовые данные должны обеспечивать демонстрацию всех основных пользовательских сценариев.

Следовательно, десяти запросов может быть недостаточно, если итоговая система предусматривает большее количество информационных операций.

В курсовой работе набор запросов должен определяться:

- требованиями системы;
- пользовательскими сценариями;
- функциями интерфейса;
- аналитическими потребностями предметной области.

Для каждого содержательного пользовательского сценария должны существовать необходимые операции получения данных.

Запросы, разработанные в настоящей работе, впоследствии могут:

- использоваться непосредственно приложением;
- входить в состав функций или процедур;
- использоваться для формирования отчётов;
- использоваться при тестировании;
- служить основой для анализа производительности.

Если в ПР № 7 реализовать только минимальные и искусственно простые запросы, при разработке пользовательского интерфейса и курсовой работы значительную часть слоя доступа к данным потребуется проектировать заново.

Поэтому для проектов, используемых в курсовой работе, рекомендуется уже сейчас формировать запросы, реально необходимые будущему приложению.

7.9 Взаимосвязь с последующими практическими работами

Результаты настоящей работы являются непосредственной основой ПР № 8.

На следующем этапе часть логики, которая в ПР № 7 представлена отдельными SQL-запросами, будет преобразована в:

- функции;
- хранимые процедуры;
- триггеры;
- более сложные серверные операции.

Тестовый набор данных также будет использоваться для проверки серверной логики.

Поэтому перед переходом к ПР № 8 необходимо обеспечить стабильность:

- физической структуры;
- основных данных;
- базовых SQL-операций.

7.10 Типичные ошибки

К наиболее распространённым ошибкам относятся:

Ошибки наполнения

- все таблицы содержат по одной записи;
- все записи практически идентичны;
- отсутствуют различные состояния объектов;
- недостаточно данных для группировки;
- отсутствие данных для отрицательных сценариев;
- ручное заполнение базы без `02_insert_data.sql`;
- нарушение связей;
- тестовые данные не соответствуют бизнес-правилам.

Ошибки запросов

- большое количество `SELECT *`;
- десять почти одинаковых запросов;
- изменение только значения `WHERE` и подсчёт запросов как различных;
- отсутствие прикладного смысла;
- `JOIN` таблиц, не имеющих смысловой связи;
- декартово произведение вследствие неправильного условия;
- использование `DISTINCT` для сокрытия ошибки соединения;

- неправильное применение агрегатных функций;
- неправильное сочетание агрегированных и неагрегированных столбцов;
- использование WHERE вместо HAVING;
- неоправданно сложный вложенный запрос;
- использование CTE там, где оно не повышает читаемость или не решает дополнительную задачу;
- запрос возвращает пустой набор из-за отсутствия тестовых данных.

Ошибки организации проекта

- фактическая база содержит записи, отсутствующие в 02_insert_data.sql;
- запросы зависят от вручную созданных данных;
- 03_queries.sql невозможно выполнить последовательно;
- отсутствуют комментарии с назначением запросов;
- структура базы была изменена, но 01_create_tables.sql не актуализирован.

7.11 Контрольные вопросы

- Для чего необходим тестовый набор данных?
- Каким требованиям должны соответствовать тестовые данные?
- Почему недостаточно заполнить каждую таблицу одной записью?
- Для чего используется WHERE?
- Для чего используется ORDER BY?
- Для чего используется JOIN?

- В чём заключается различие между INNER JOIN и внешним соединением?
- Для чего применяются агрегатные функции?
- Для чего используется GROUP BY?
- В чём заключается различие между WHERE и HAVING?
- Что представляет собой вложенный запрос?
- Что представляет собой CTE?
- В каких случаях применение CTE повышает качество запроса?
- Почему SELECT * не рекомендуется использовать как основу прикладных запросов?
- Что может приводить к появлению неожиданных дубликатов после JOIN?
- Почему запрос должен иметь прикладную постановку задачи?
- Что понимается под воспроизводимостью тестовой базы?
- Каким образом разработанные запросы будут использоваться в последующих практических работах?

7.12 Критерии приёмки работы

Практическая работа может быть зачтена при одновременном выполнении следующих условий:

- структура базы соответствует защищённой ПР № 6;
- создан воспроизводимый файл 02_insert_data.sql;
- основные таблицы содержат тестовые данные;
- база содержит достаточный объём содержательных записей;

- основная операционная сущность представлена не менее чем 10 экземплярами;
- совокупный тестовый набор содержит не менее 50 содержательных записей либо согласованный эквивалент;
- данные позволяют проверять различные состояния и сценарии;
- сформулировано не менее 10 самостоятельных информационных задач;
- реализовано не менее 10 содержательных SQL-запросов;
- присутствует фильтрация;
- присутствует сортировка;
- присутствуют соединения;
- имеется соединение не менее трёх таблиц;
- используются агрегатные функции;
- используется GROUP BY;
- используется HAVING;
- реализован вложенный запрос или CTE;
- имеется как минимум один комплексный запрос;
- запросы соответствуют предметной области;
- результаты выполнения проверены;
- SQL-файлы позволяют воспроизвести полученный результат;
- обучающийся способен объяснить и модифицировать разработанные запросы.

7.13 Срок выполнения и сдачи

Срок выполнения и защиты практической работы определяется календарным графиком практических занятий.

Практическая работа № 7 должна быть завершена до начала разработки серверной бизнес-логики.

К моменту перехода к ПР № 8 база данных должна содержать стабильную физическую структуру, репрезентативный тестовый набор и проверенный набор основных SQL-операций.